

Lecture 27 — Asynchronous I/O with cURL

Jeff Zarnett

2026-08-13

Using cURL Asynchronously

We've already seen that network communication is a great example of a way that you could use asynchronous I/O. You can start a network request and move on to creating more without waiting for the results of the first one. For requests to different recipients, it certainly makes sense to do this. And yet, we've seen that while socket programming is important and necessary, for a lot of situations we prefer to use cURL. And we can use this in an asynchronous way as well.

The cURL multi interface has a lot of similarities with the regular cURL interface. It's been a little while since we went over it, so let's recap what we did before. Remember from earlier the example we did that was modified from <https://curl.haxx.se/libcurl/c/https.html> (i.e., the official docs):

```
#include <stdio.h>
#include <curl/curl.h>

int main( int argc, char** argv ) {
    CURL *curl;
    CURLcode res;

    curl_global_init(CURL_GLOBAL_DEFAULT);

    curl = curl_easy_init();
    if( curl ) {
        curl_easy_setopt(curl, CURLOPT_URL, "https://example.com/" );
        res = curl_easy_perform( curl );

        if( res != CURLE_OK ) {
            fprintf(stderr, "curl_easy_perform() failed: %s\n", curl_easy_strerror(res));
        }
        curl_easy_cleanup(curl);
    }

    curl_global_cleanup();
    return 0;
}
```

In the previous example, the call to `curl_easy_perform()` is blocking and we wait for the curl execution to take place. We want to change that! The tool for this is the “multi handle” - this is a structure that lets us have more than one curl easy handle. And rather than waiting, we can start them and then check on their progress.

There are still the global initialization and cleanup functions. The structure for the new multi-handle type is `CURLM` (instead of `CURL`) and it is initialized with the `curl_multi_init()` function.

Once we have a multi handle, we can add easy handles – however many we need – to the multi handle. Creation of the easy handle is the same as it is when being used alone - use `curl_easy_init()` to create it and then we can set however many options on this we need. Then, we add the easy handle to the multi handle with `curl_multi_add_handle( CURLM* cm, CURL* eh )`.

Once we have finished putting all the easy handles into the multi handle, we can dispatch them all at once with `curl_multi_perform( CURLM* cm, int* still_running )`. The second parameter is a pointer to an integer that is updated with the number of the easy handles in that multi handle that are still running. If it's down to 0, then we know that they are all done. If it's nonzero it means that some of them are still in progress.

This does mean that we're going to call `curl_multi_perform()` more than once. Doing so doesn't restart or interfere with anything that was already in progress – it just gives us an update on the status of what's going on. We can check as often as we'd like, but the intention is of course to do something useful while the asynchronous I/O request(s) are going on. Otherwise, why not make it synchronous?

Suppose we've run out of things to do though. What then? Well, we can wait, if we want, using `curl_multi_wait( CURLM *multi_handle, struct curl_waitfd extra_fds[], unsigned int extra_nfds, int timeout_ms, int *numfds )`. This function will block the current thread until something happens (some event occurs).

The first parameter is the multi handle, which makes sense. The second parameter is a structure of extra file descriptors you can wait on (but we will always want this to be NULL in this course) and the third parameter is the count (the size of the provided array) which would also be zero here. Then the second-last parameter is a maximum time to wait. The last parameter is a pointer that will be updated with the actual number of “interesting” events that occurred (interesting is the word used in the specifications, and what it means is mysterious). For a simple use case you can ignore most of the parameters and just wait for something to happen and go from there.

In the meantime though, the perform operations are happening, and so are whatever callbacks we have set up (if any). And as the I/O operation moves through its life cycle, the state of the easy handle is updated appropriately. Each easy handle has an associated status message as well as a return code.

Why both? Well - one is about what the status of the request is. The message could be, for example “done”, but does that mean finished with success or finished with an error? The second one tells us about that. We can ask about the status of the request using `curl_multi_info_read( CURLM* cm, int* msgs_left )`. This returns a pointer to information about the “next” easy handle, if there is one. The return value is a pointer to a struct of type `CURLMsg`. Along side this, the parameter `msgs_left` is updated to say how many messages remain (so you don't have to remember or know in advance, really).

We will therefore check the `CURLMsg` message to see what happened and make sure all is well. If our message that we got back with the info read is called `m`, what we are looking for is that the `m->msg` is equal to `CURLMSG_DONE` – request completed. If not, this request is still in progress and we aren't ready to evaluate whether it was successful or not. If there are more handles to look at, we should go on to the next. If it is done, we should look at the return code and the result, in `m->data.result`. If it is `CURLE_OK` then everything succeeded. If it's anything else, it indicates an error.

When a handle has finished, you need to remove it from the multi handle. A pointer to it is inside the `CURLMsg` under `m->easy_handle`. It is removed with `curl_multi_remove_handle( CURLM* cm, CURL* eh )`. Once removed, it should be cleaned up like normal with `curl_easy_cleanup( CURL* eh )`.

There is of course the corresponding cleanup function `curl_multi_cleanup( CURLM * cm )` for the multi handle when we are done with all the easy handles inside. The last step, as before, is to use the global cleanup function. After that we are done.

Let's consider the following code example by Clemens Gruber [Gru13], with slight modifications for compactness, formatting, and to remember the cleanup. This example puts together all the things we talked about in one compact code segment. Here, the callback does nothing, but that's okay – it's just to show what you could do with it.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <curl/multi.h>

#define MAX_WAIT_MSECS 30*1000 /* Wait max. 30 seconds */

const char *urls[] = {
    "http://www.microsoft.com",
    "http://www.yahoo.com",
    "http://www.wikipedia.org",
    "http://slashdot.org"
```

```

};
#define CNT 4

size_t cb(char *d, size_t n, size_t l, void *p) {
    /* take care of the data here, ignored in this example */
    return n*l;
}

void init( CURLM *cm, int i ) {
    CURL *eh = curl_easy_init();
    curl_easy_setopt( eh, CURLOPT_WRITEFUNCTION, cb );
    curl_easy_setopt( eh, CURLOPT_HEADER, 0L );
    curl_easy_setopt( eh, CURLOPT_URL, urls[i] );
    curl_easy_setopt( eh, CURLOPT_PRIVATE, urls[i] );
    curl_easy_setopt( eh, CURLOPT_VERBOSE, 0L );
    curl_multi_add_handle( cm, eh );
}

int main( int argc, char** argv ) {
    CURLM *cm = NULL;
    CURL *eh = NULL;
    CURLMsg *msg = NULL;
    CURLcode return_code = 0;
    int still_running = 0;
    int msgs_left = 0;
    long http_status_code;
    const char *szUrl;

    curl_global_init( CURL_GLOBAL_ALL );
    cm = curl_multi_init( );

    for ( int i = 0; i < CNT; ++i ) {
        init( cm, i );
    }

    curl_multi_perform( cm, &still_running );

    do {
        int numfds = 0;
        int res = curl_multi_wait( cm, NULL, 0, MAX_WAIT_MSECS, &numfds );
        if( res != CURLM_OK ) {
            fprintf( stderr, "error:_curl_multi_wait()_returned_%d\n", res );
            return EXIT_FAILURE;
        }
        curl_multi_perform( cm, &still_running );
    } while( still_running );

    while ( ( msg = curl_multi_info_read( cm, &msgs_left ) ) ) {
        if ( msg->msg == CURLMSG_DONE ) {
            eh = msg->easy_handle;

            return_code = msg->data.result;
            if ( return_code != CURLE_OK ) {
                fprintf( stderr, "CURL_error_code:_%d\n", msg->data.result );
                curl_multi_remove_handle( cm, eh );
                curl_easy_cleanup( eh );
                continue;
            }

            // Get HTTP status code
            http_status_code = 0;
            szUrl = NULL;

            curl_easy_getinfo( eh, CURLINFO_RESPONSE_CODE, &http_status_code );
            curl_easy_getinfo( eh, CURLINFO_PRIVATE, &szUrl );

            if( http_status_code == 200 ) {
                printf( "200_OK_for_%s\n", szUrl );
            } else {
                fprintf( stderr, "GET_of_%s_returned_http_status_code_%d\n", szUrl, http_status_code );
            }
        }
    }
}

```

```

    }

    curl_multi_remove_handle( cm, eh );
    curl_easy_cleanup( eh );
} else {
    fprintf( stderr, "error:_after_curl_multi_info_read(),_CURLMsg=%d\n", msg->msg );
}
}
curl_multi_cleanup( cm );
curl_global_cleanup();
return 0;
}

```

You may wonder about re-using an easy handle rather than removing and destroying it and making a new one. The official docs say that you can re-use one, but you have to remove it from the multi handle and then re-add it, presumably after having changed anything that you want to change about that handle. In the example above, handles are not reused, but if we wanted to we would reset whatever parameters, if any, and then re-add it with `curl_multi_add_handle`.

Because a handle could be replaced with another one (or the same one), you could have a situation where there are constantly handles in progress and you might never be at a situation where there are no messages left. And that is okay.

In this example all requests had the same (useless) callback, but of course you could have different callbacks for different easy handles if you wanted them to do different things.

How well does this scale? The developer claims that you can have multiple thousands of connections in a single multi handle¹. And 60k ought to be enough for anyone!

I enjoy pain! You can use cURL with `select()` if you wish, although it comes with an anti-recommendation: I think you shouldn't do it. But you can if you want. In some ways, cURL does make things less painful because it does some of the grunt work for you.

The first part of using cURL with `select` is of course setting up your multi handle and adding all of the easy handles to that. Once you've done that, you can ask cURL to prep things for you using `curl_multi_fdset`:

```

CURLMcode curl_multi_fdset( CURLM *multi_handle, fd_set *read_fd_set, fd_set *write_fd_set, fd_set *exc_fd_set,
    int *max_fd );

```

It takes as arguments, first of all, the multi handle, and then pointers to the `fd_sets`. It's the responsibility of the caller to set these to zero with `FD_ZERO` before calling this. And then finally, the function is kind enough to tell you what the maximum file descriptor is. Which is convenient! You can give in `NULL` for the sets if you like. So, a super simple example:

```

fd_set fdr;
fd_set fdw;
fd_set fde;
FD_ZERO( &fdr );
FD_ZERO( &fdw );
FD_ZERO( &fde );
int maxfd = -1;

CURLMcode res = curl_multi_fdset( cm, &fdr, &fdw, &fde, &maxfd );

```

This fills in the values of `fdr` and `maxfd` for us, which is some of what, but not everything that, we need. When calling `select` we still have to put a +1 on the max file descriptor, but that is no big deal. The missing element is the timeout. For that there is another function:

```

CURLMcode curl_multi_timeout( CURLM *multi_handle, long *timeout );

```

¹See this post from the mailing list: <https://curl.haxx.se/mail/lib-2011-11/0078.html>

Unfortunately, though, this returns a long and not one of the time structures we've seen previously. So you have to do the math yourself. Why could this not be in the spec... or a standard function. This is certainly the sort of thing that you would want to write into a function to convert, so you don't do it by hand every single time. See the example below of how to convert it, as written in the documentation:

```
struct timeval timeout;
long timeo;

curl_multi_timeout( cm, &timeo );
if(timeo < 0) {
    /* no set timeout, use a default */
    timeo = 980;
}

timeout.tv_sec = timeo / 1000;
timeout.tv_usec = (timeo % 1000) * 1000;
```

But we aren't ready to call `select` yet – we haven't started any of the transfers that use the sockets. For that we use `curl_multi_perform` just as in the earlier example. That starts the transfers. If you follow the cURL examples, they do that step before calling any of the setup functions that prepare the arguments for `select`, because you can do this setup while the transfers are in progress. Or you can do it after the setup, but it does have to be before the call to `select`. Otherwise `select` is waiting for a thing that won't happen: if there are no transfers in progress then data won't become available.

When we wake up, it is because something happened, and we still use `curl_multi_info_read` to find out what's going on with a given easy handle and find out its status and also find out how many are still running. As with the earlier code where we used `select` on sockets directly, we have to remember to reset (repopulate) the values that are passed to that function before the next iteration of the loop.

We still have to remember to clean everything up: both the cURL related stuff and anything related to `select`.

One of the other problems with `select` is that you are limited in the number of file descriptors because you could overflow the bitmasks. But even if you don't, this really does not scale, so you would be better off using the regular curl multi call. As far as I can see it's better in pretty much every way.

Maybe not this much pain... Can we use cURL with `poll`? Generally, no. The lead developer of cURL himself says that the way forward here is to use `curl_multi_wait` instead of the `select` interface and that there isn't really a `poll` interface for it². Long story short: don't.

References

[Gru13] Clemens Gruber. libcurl multi interface example, 2013. Online; accessed 30-October-2018. URL: <https://gist.github.com/clemensg/4960504>.

²See this 2013 post from the mailing list: <https://curl.haxx.se/mail/lib-2013-03/0188.html>